

49– Repository Pattern

Repository Pattern is an abstraction of Data Access Layer. It hides the details of how exactly the data is saved or retrieved from the underlying data source. The details of how the data is stored and retrieved is in the respective repository.

CRUD operations = Create, Retrieve, Update, Delete.

В най-простата си форма, Repository Patterns съдържа 2 участника: интерфейс + клас, който имплементира този интерфейс.

Към момента, ние работим с този

Employee.cs тип, затова сме нарекли интерфейса IEmployeeRepository.

ако работим с Product.cs тип, ще го наречем IProductRepository;

съответно за Customer.cs тип => ICustomerRepository.

```
public interface IEmployeeRepository
{
    2 references
    Employee GetEmployee (int Id); // method GetEmployee
    3 references
    IEnumerable<Employee> GetAllEmployee();
    2 references
    Employee Add(Employee employee);
}
```

Към момента нямаме всички CRUD операции. Искаме да добавим Update + Delete.

```
Employee Add(Employee employee);
Employee Update(Employee employee); Tab to accept
```

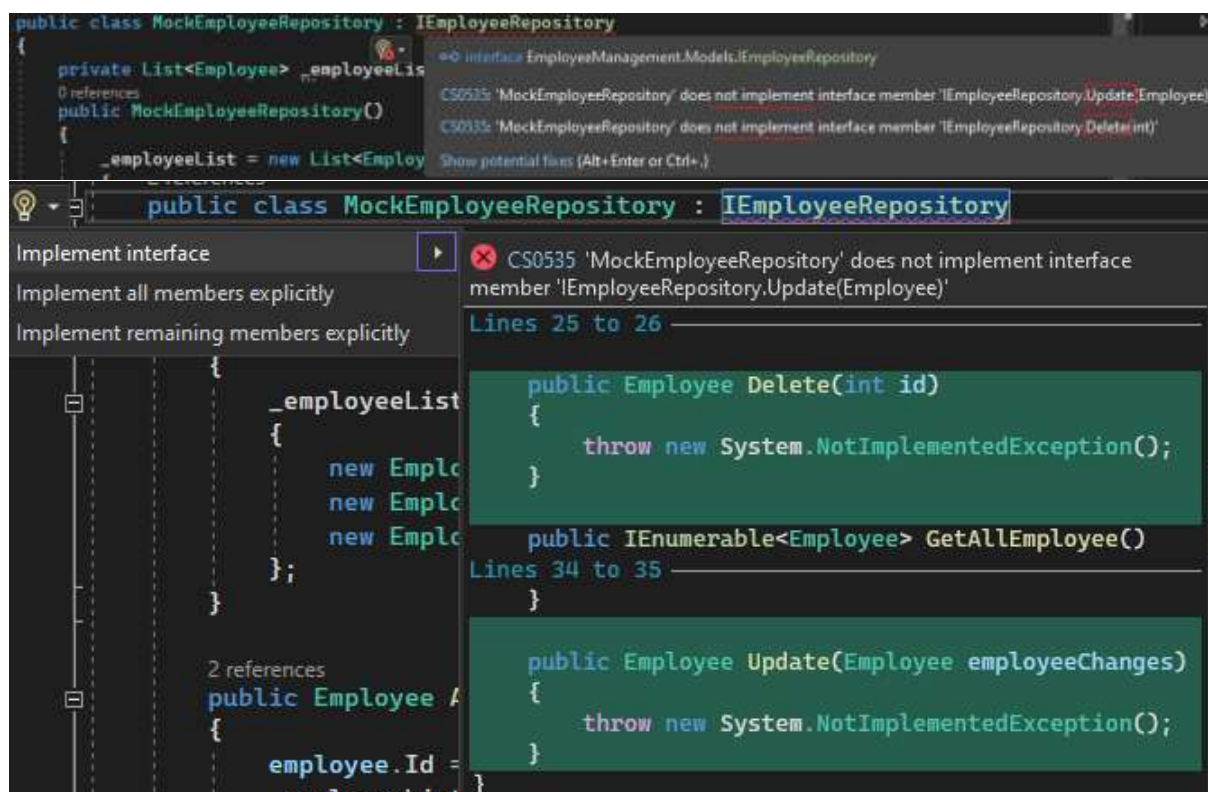
Intellisense предлага последователно Update, Delete

```
public interface IEmployeeRepository
{
    2 references
    Employee GetEmployee (int Id); // Read
    3 references
    IEnumerable<Employee> GetAllEmployee();
    2 references
    Employee Add(Employee employee); // Create
    0 references
    Employee Update(Employee employeeChanges); // Update
    0 references
    Employee Delete(int id); // Delete
}
```

IEmployeeRepository interface supports the following operations:

- * Get all the employees
- * Get a single employee by id
- * Add a new employee
- * Update an employee
- * Delete an employee

Важният момент тук е, че Repository интерфейсът съдържа само операции, които са поддържани от този Repository. Той не съдържа детайли КАК тези операции се изпълняват. Тези подробности се намират в съответния Repository class, в нашия случай MockEmployeeRepository.cs



private List<Employee> _employeeList;

Първо искаме от private field _employeeList да изтрием елемент, затова извикваме FirstOrDefault, който намира елемента по подаденото id.

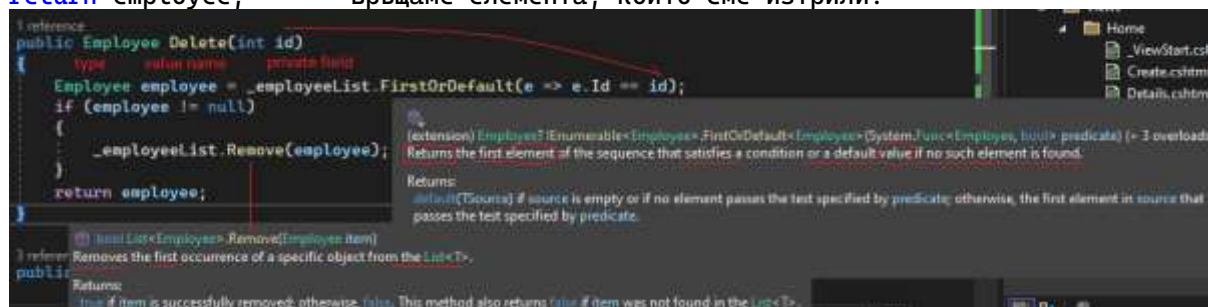
_employeeList.FirstOrDefault(e => e.Id == id);

После запазваме елемента, който е резултат от кода, като го съхраняваме в променлива, която наричаме employee, от тип Employee.

if (employee != null) – ако сме намерили такъв елемент

_employeeList.Remove(employee); – го премахваме от листа (частното поле).

return employee; – връщаме елемента, който сме изтрили.



```
public Employee Delete(int id)
{
    Employee employee = _employeeList.FirstOrDefault(e => e.Id == id);
    if (employee != null)
    {
        _employeeList.Remove(employee);
    }
    return employee;
}
```

За Update кодът е почти същият, затова го копираме и поставяме за леки промени – прихващаме Id на новата променлива, която е въведена employeeChanges. Нейните променливи се съхраняват в employee, като тя приема съответните стойности на въведените.

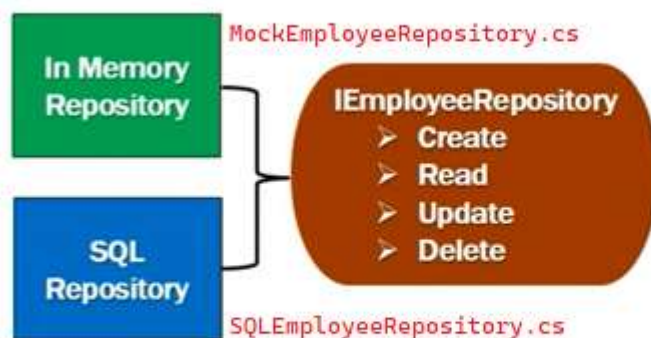
```

public Employee Update(Employee employeeChanges)
{
    Employee employee = _employeeList.FirstOrDefault(e => e.Id == employeeChanges.Id);
    if (employee != null)
    {
        employee.Name = employeeChanges.Name;
        employee.Email = employeeChanges.Email;
        employee.Department = employeeChanges.Department;
    }
}

public Employee Update(Employee employeeChanges)
{
    Employee employee = _employeeList.FirstOrDefault(
e => e.Id == employeeChanges.Id);
    if (employee != null)
    {
        employee.Name = employeeChanges.Name;
        employee.Email = employeeChanges.Email;
        employee.Department = employeeChanges.Department;
    }
    return employee;
}

```

Repository Pattern in ASP.NET Core



```

public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository;
}

```

Искаме вместо в паметта, данните да се съхраняват в SqlServer, затова добавяме нов клас.

Models folder -> Add Class -> SQLEmployeeRepository.cs

Искаме той да имплементира интерфейса IEmployeeRepository, затова го извикваме:

```
SQLEmployeeRepository.cs* | EmployeeRepository.cs* | Startup.cs* | MockEmployeeRepository.cs
EmployeeManagement | EmployeeManagement.Models | SQLEmployeeRepository

0 references
public class SQLEmployeeRepository : IEmployeeRepository
{
    2 references
    public Employee Add(Employee employee)
    {
        throw new NotImplementedException();
    }

    1 reference
    public Employee Delete(int id)
    {
        throw new NotImplementedException();
    }

    3 references
    public IEnumerable<Employee> GetAllEmployee()
    {
        throw new NotImplementedException();
    }

    2 references
    public Employee GetEmployee(int Id)
    {
        throw new NotImplementedException();
    }
}
```

За да може този клас да работи с данни от SQL Server DB, ни е необходима връзка с DbContext class, която имплементирахме като AppDbContext.cs по-рано.

```
SQLEmployeeRepository.cs* | AppDbContext.cs | EmployeeRepository.cs* | MockEmployeeRepository.cs
EmployeeManagement | EmployeeManagement.Models | AppDbContext | Employees

using Microsoft.EntityFrameworkCore;

namespace EmployeeManagement.Models
{
    3 references
    public class AppDbContext : DbContext
    {
        0 references
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {
        }

        0 references
        public DbSet<Employee> Employees { get; set; }
    }
}
```

За да инжектираме AppDbContext в SQLEmployeeRepository, ни е необходим ctor:


```

1 reference
public class SQLEmployeeRepository : IEmployeeRepository
{
    private readonly AppDbContext context; ← auto field
    0 references
    public SQLEmployeeRepository(AppDbContext context) ← inject class, name of parameter
    {
        this.context = context; ← ctrl.
    }
}

```

След това ни трябва имплементации за всички CRUD methods.

```

SQLEmployeeRepository.cs  MockEmployeeRepository.cs
EmployeeManagement.Models.SQLEmployeeRepository  Delete(int id)
private readonly AppDbContext context;

0 references
public SQLEmployeeRepository(AppDbContext context)
{
    this.context = context;
}

2 references
public Employee Add(Employee employee)
{
    context.Employees.Add(employee); ← public DbSet<Employee> Employees { get; set; }
    context.SaveChanges(); // method for DB
    return employee;
}

1 reference
public Employee Delete(int id)
{
    Employee employee = context.Employees.Find(id); // Find(x => x.Id == id).Delete();
    if(employee != null) ← if we found it
    {
        context.Employees.Remove(employee);
        context.SaveChanges();
    }
    return employee;
}

public IEnumerable<Employee> GetAllEmployee()
{
    return context.Employees;
}

2 references
public Employee GetEmployee(int Id)
{
    return context.Employees.Find(Id);
}

1 reference
public Employee Update(Employee employeeChanges)
{
    var employee = context.Employees.Attach(employeeChanges);
    employee.State = Microsoft.EntityFrameworkCore.EntityState.Modified;
    context.SaveChanges();
    return employeeChanges;
}

```

enum Microsoft.EntityFrameworkCore.EntityState
The state in which an entity is being tracked by a context.

```
public class SQLEmployeeRepository : IEmployeeRepository
```

```

{
    private readonly AppDbContext context;

    public SQLEmployeeRepository(AppDbContext context)
    {
        this.context = context;
    }

    public Employee Add(Employee employee)
    {
        context.Employees.Add(employee);
        context.SaveChanges(); // method for DB
        return employee;
    }

    public Employee Delete(int id)
    {
        Employee employee = context.Employees.Find(id);
//Find(x => x.Id == id).Delete();
        if(employee != null)
        {
            context.Employees.Remove(employee);
            context.SaveChanges();
        }
        return employee;
    }

    public IEnumerable<Employee> GetAllEmployee()
    {
        return context.Employees;
    }

    public Employee GetEmployee(int Id)
    {
        return context.Employees.Find(Id);
    }

    public Employee Update(Employee employeeChanges)
    {
        var employee = context.Employees.Attach(employeeChanges);
        employee.State = Microsoft.EntityFrameworkCore.EntityState.Modified;
        context.SaveChanges();
        return employeeChanges;
    }
}

```

}

Repository Pattern in ASP.NET Core



```
public void ConfigureServices(IServiceCollection services)
{
    // Other Code...
    services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
}
```

Към момента имаме 2 имплементации на Repository – Mock (in-memory) и SQL. В HomeController не е указано коя да се използва, виждаме само `private readonly IEmployeeRepository _employeeRepository;` Коя да избере контролерът, се задава в Startup.cs -> ConfigureServices

```
oContext.cs  Startup.cs  X  SQLEmployeeRepository.cs  IEmployeeRepository.cs*  MockEmployeeRepository.cs
EmployeeManagement.Startup  ConfigureServices(IServiceCollection services)
0 references
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>((
        options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));

    services.AddMvc(option => option.EnableEndpointRouting = false)
        .AddXmlSerializerFormatters();
    services.AddTransient<IEmployeeRepository, MockEmployeeRepository>();
}
```

Към момента сме задали Mock.

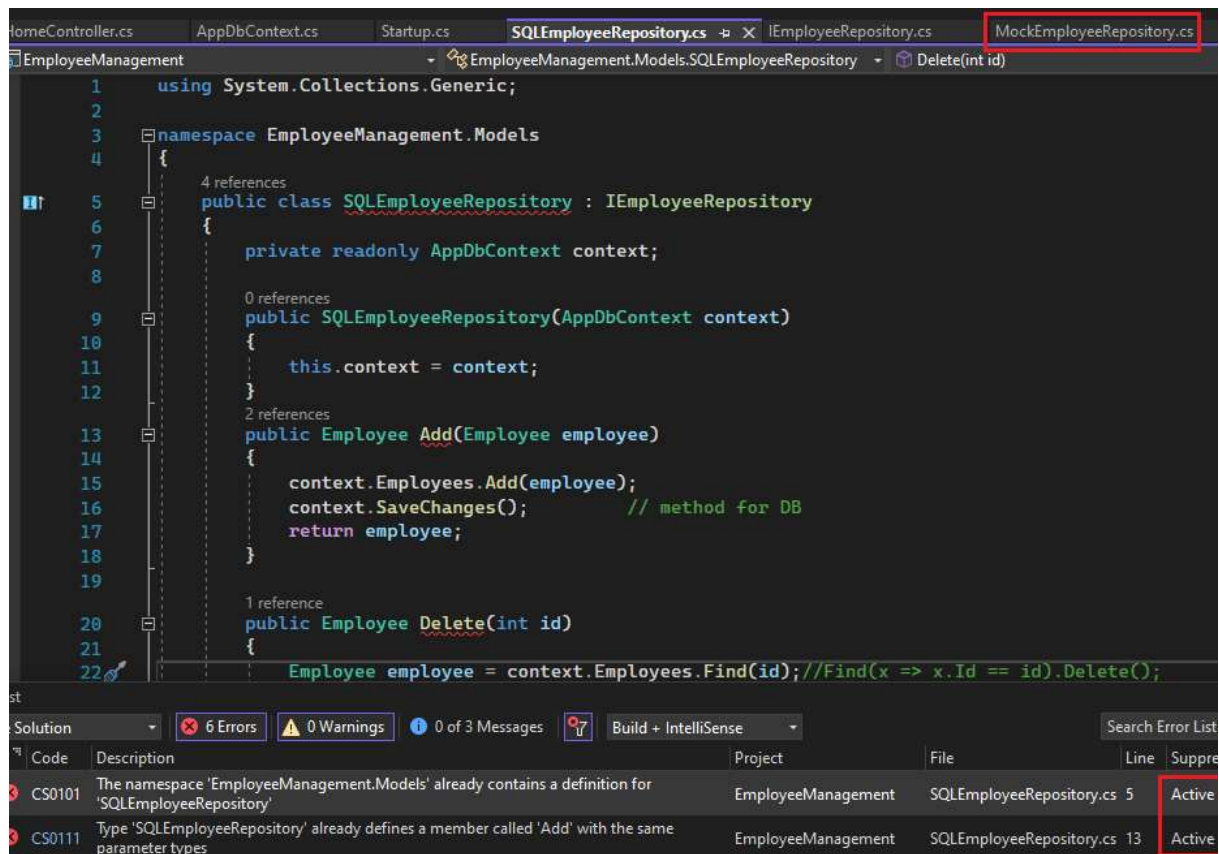
```
services.AddTransient<IEmployeeRepository, SQLEmployeeRepository>();
```

С промяна в тази една линия код, нашето приложение вече използва БД. Това е силата и гъвкавостта на dependency injection.

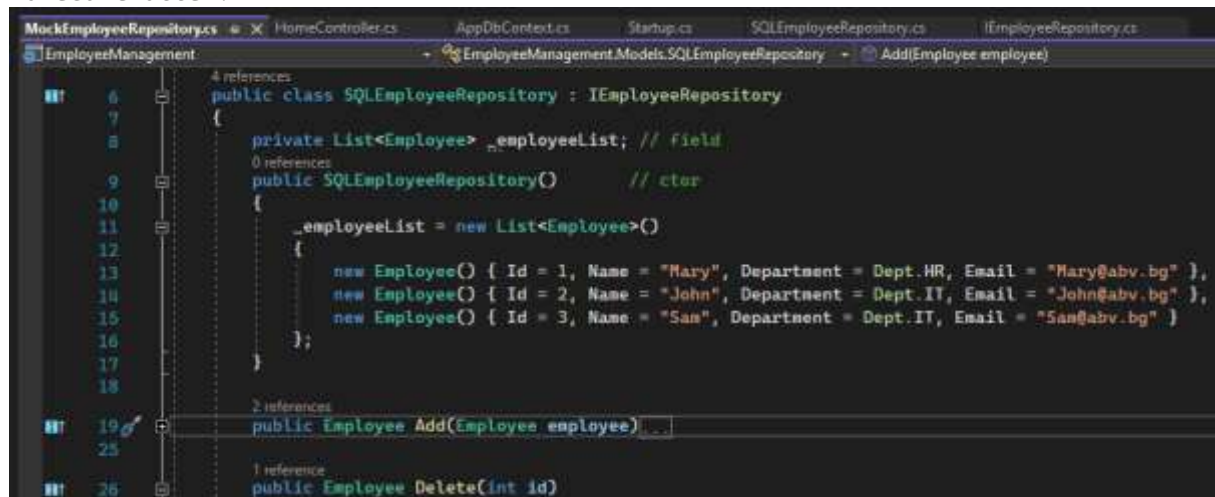
```
services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
```

Променяме и връзката между двата Repository на Scoped, за регистрация на нашата услуга service, защото искаме инстанцията на IEmployeeRepository class да бъде активна и достъпна по време на целия обхват на дадена HTTP заявка. Когато нова заявка бъде подадена, искаме да се създаде нова инстанция, която от своя страна също да бъде активна и достъпна по време на целия обхват на тази HTTP заявка.

Предимствата са, че кодът е много по-изчистен и гъвкав, особено, когато се наложи използване на OracleDB. При тестване е много по-удачно да се смени и използва in-memory, преди да се включи реална BD, което също става с лесна промяна в един ред код.



Излязоха 6 грешки, които помислих, че са от отворения Моск, но не са. Маркира повторените CRUD methods в двата файла Моск и SQL като дублиращи се, въпреки, че вече използвам само SQL навсякъде, а Моск не е упоменат никъде, дори в HomeController.



localhost:24077/Home/Create

List Create

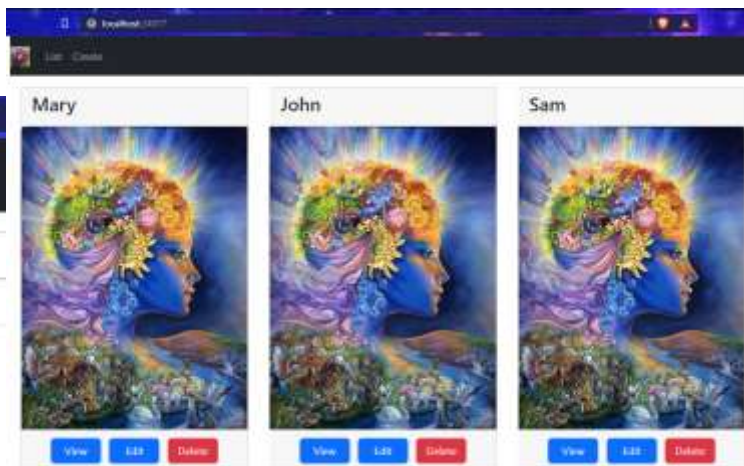
Name: David

Office Email: david@abv.bg

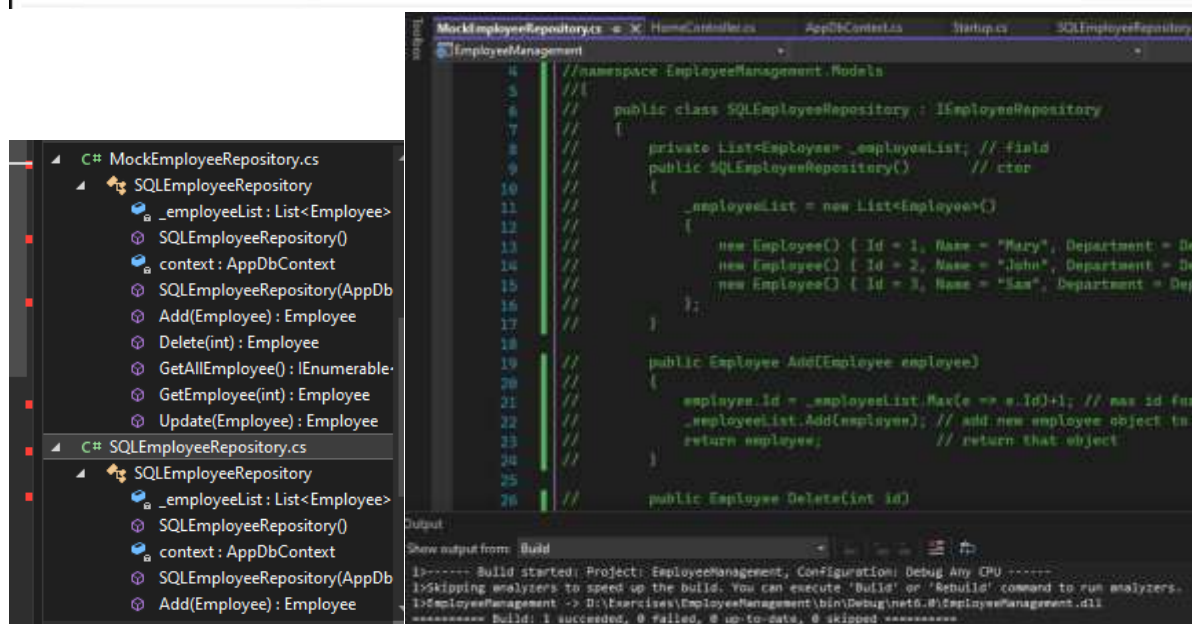
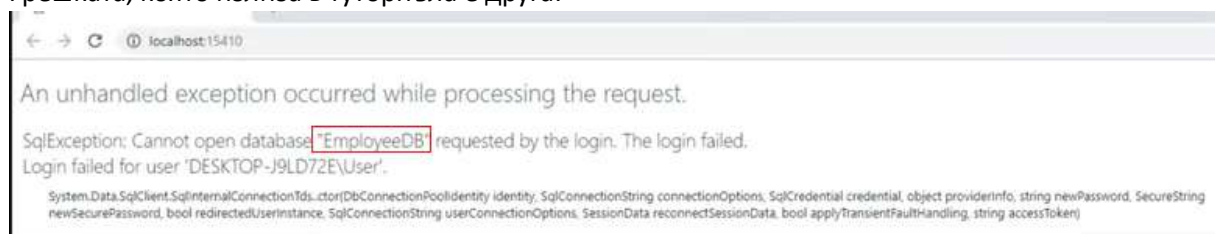
Department: IT

Create

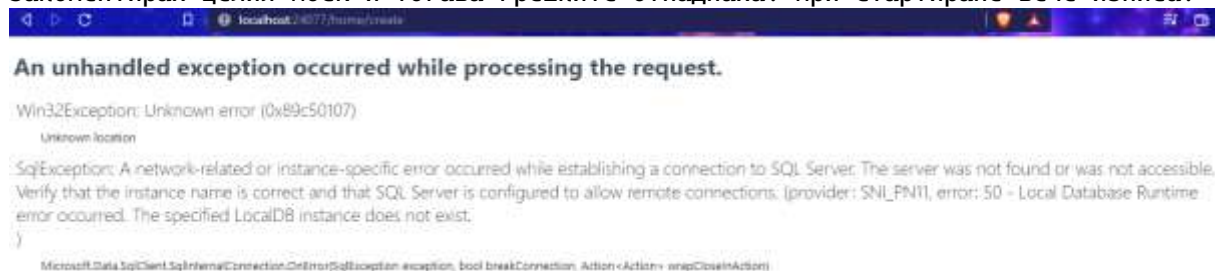
Total Employees Count = 4



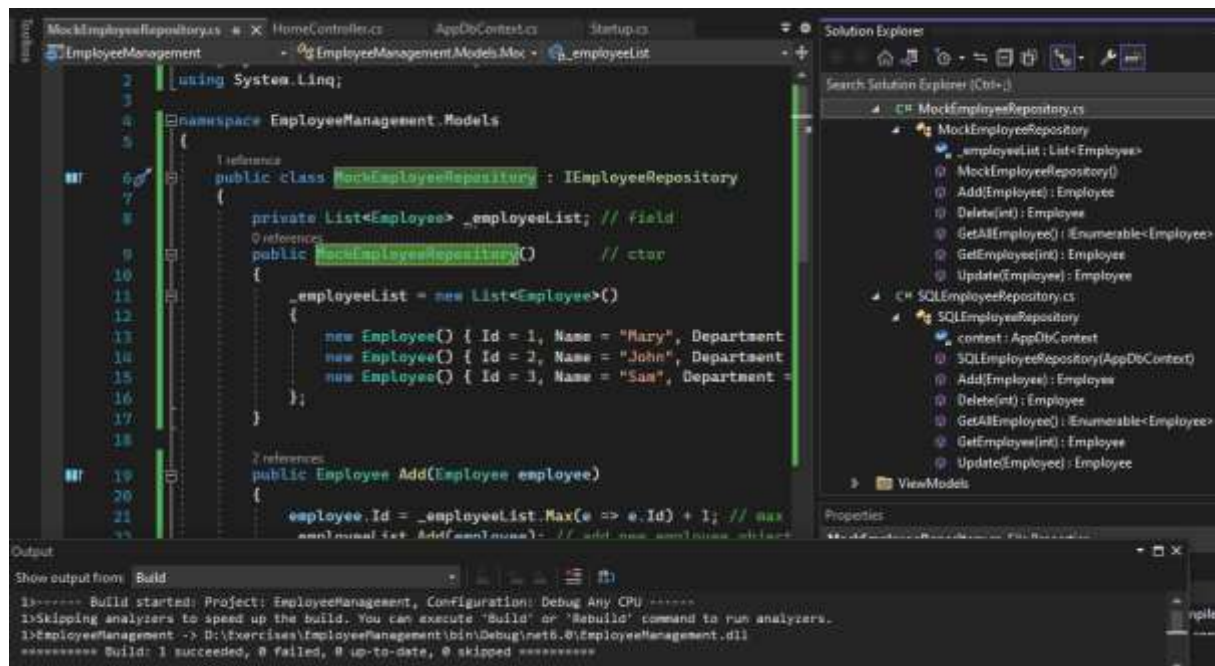
Създава нов служител, но не го съхранява нито в In-memory, защото никъде не се извиква Mock, нито с SQL, защото EmployeeDB не е създадена.
Грешката, която излиза в туториъла е друга:



Закоментирах целия Mock и тогава грешките отпаднаха. При стартиране вече изписа:



Като отворих Mock, се оказа, че на 2 места автоматично е бил сменен с SQL



Изписаната грешка, че локална БД липсва е обяснима – все още не сме я създали.